
FitTrack:Strava Fitness Data Analytics Case-Study An Analysis of User Activity

Name: Mannepalli Abhilash

Objective: To analyze a dataset from a fitness tracking application to uncover user habits, patterns, and relationships between physical activity, sleep, and calorie expenditure. This report details the process of data cleaning, merging, analysis, and visualization.

Problem Statement:

Bellabeat, a high-tech wellness company focused on women's health, aimed to understand how consumers are using smart fitness devices to identify new growth opportunities and refine its marketing strategy. The core challenge was to analyze existing smart device data to gain actionable insights into user behavior and inform future marketing decisions.

Summary of Analysis:

This project involved a comprehensive analysis of 18 Fitbit smart device data files. The process included robust data loading, extensive cleaning and preprocessing (handling date formats, missing values, and data quality issues like sedentary periods), and merging various datasets (daily activity, sleep, calories, intensities). Through SQL queries and a series of detailed Tableau dashboard visualizations, key insights were uncovered: users exhibited fluctuating daily and weekly activity levels (with lower engagement on Sundays), distinct hourly peaks in calorie burn and intensity, and varied sleep patterns. A significant portion of the data indicated prolonged sedentary times for users. Based on these findings, high-level content recommendations were formulated for Bellabeat, focusing on promoting micro-breaks, weekend activity challenges, emphasizing light activity benefits, personalized calorie insights, and improved sleep health education, all aimed at empowering women and fostering sustained user engagement.

1. Setup and Data Loading

Python

```
import pandas as pd
import matplotlib.pyplot as plt
```

```

import seaborn as sns
import numpy as np
from datetime import datetime, timedelta
import os
loaded_csv= [
'dailyActivity_merged.csv',
'dailyCalories_merged.csv',
'dailyIntensities_merged.csv',
'dailySteps_merged.csv',
'heartrate_seconds_merged.csv',
'hourlyCalories_merged.csv',
'hourlyIntensities_merged.csv',
'hourlySteps_merged.csv',
'minuteCaloriesNarrow_merged.csv',
'minuteCaloriesWide_merged.csv',
'minuteIntensitiesNarrow_merged.csv',
'minuteIntensitiesWide_merged.csv',
'minuteMETsNarrow_merged.csv',
'minuteSleep_merged.csv',
'minuteStepsNarrow_merged.csv',
'minuteStepsWide_merged.csv',
'sleepDay_merged.csv',
'weightLogInfo_merged.csv'
]

datasets = r'/Users/abhillaahsh/Downloads/STRAVA FITNESS APP/datafiles'

full_paths = [os.path.join(datasets, fname) for fname in loaded_csv]

# Dictionary to store DataFrames
dataframes = {}

print(" Loading datasets:")
print("=" * 60)

# Load each CSV file
for path in full_paths:
    try:
        # Get simple name for dictionary key
        name = os.path.basename(path).replace('.csv', '')

        # Read CSV file
        df = pd.read_csv(path)
        dataframes[name] = df

        # Print status
        print(f" {name: <35} | Rows: {df.shape[0]:<6} | Columns: {df.shape[1]}")

```

```

except Exception as e:
    print(f" Error loading {os.path.basename(path)}: {str(e)}")

print("=" * 60)
print(f"Successfully loaded {len(dataframes)}/{len(full_paths)} datasets\n")

```

Loading Output:

Loading datasets:

```

=====
dailyActivity_merged      | Rows: 940   | Columns: 15
dailyCalories_merged      | Rows: 940   | Columns: 3
dailyIntensities_merged   | Rows: 940   | Columns: 10
dailySteps_merged         | Rows: 940   | Columns: 3
heartrate_seconds_merged  | Rows: 2483658 | Columns: 3
hourlyCalories_merged     | Rows: 22099 | Columns: 3
hourlyIntensities_merged  | Rows: 22099 | Columns: 4
hourlySteps_merged        | Rows: 22099 | Columns: 3
minuteCaloriesNarrow_merged | Rows: 1325580 | Columns: 3
minuteCaloriesWide_merged | Rows: 21645  | Columns: 62
minuteIntensitiesNarrow_merged | Rows: 1325580 | Columns: 3
minuteIntensitiesWide_merged | Rows: 21645  | Columns: 62
minuteMETsNarrow_merged   | Rows: 1325580 | Columns: 3
minuteSleep_merged        | Rows: 188521 | Columns: 4
minuteStepsNarrow_merged  | Rows: 1325580 | Columns: 3
minuteStepsWide_merged    | Rows: 21645  | Columns: 62
sleepDay_merged           | Rows: 413    | Columns: 5
weightLogInfo_merged      | Rows: 67     | Columns: 8
=====
Successfully loaded 18/18 datasets

```

2. All files loaded

```

print("\n All files loaded (if found)")
print("DataFrames available in 'all_dataframes' dictionary:")
for df_name, df in dataframes.items():
    print(f"- {df_name} (columns: {df.columns.tolist()})")

```

Loaded Files Data Output:

All files loaded (if found)
DataFrames available in 'all_dataframes' dictionary:

- dailyActivity_merged (columns: ['Id', 'ActivityDate', 'TotalSteps', 'TotalDistance', 'TrackerDistance', 'LoggedActivitiesDistance', 'VeryActiveDistance', 'ModeratelyActiveDistance', 'LightActiveDistance', 'SedentaryActiveDistance', 'VeryActiveMinutes', 'FairlyActiveMinutes', 'LightlyActiveMinutes', 'SedentaryMinutes', 'Calories'])
- dailyCalories_merged (columns: ['Id', 'ActivityDay', 'Calories'])
- dailyIntensities_merged (columns: ['Id', 'ActivityDay', 'SedentaryMinutes', 'LightlyActiveMinutes', 'FairlyActiveMinutes', 'VeryActiveMinutes', 'SedentaryActiveDistance', 'LightActiveDistance', 'ModeratelyActiveDistance', 'VeryActiveDistance'])
- dailySteps_merged (columns: ['Id', 'ActivityDay', 'StepTotal'])
- heartrate_seconds_merged (columns: ['Id', 'Time', 'Value'])
- hourlyCalories_merged (columns: ['Id', 'ActivityHour', 'Calories'])
- hourlyIntensities_merged (columns: ['Id', 'ActivityHour', 'TotalIntensity', 'AverageIntensity'])
- hourlySteps_merged (columns: ['Id', 'ActivityHour', 'StepTotal'])
- minuteCaloriesNarrow_merged (columns: ['Id', 'ActivityMinute', 'Calories'])
- minuteCaloriesWide_merged (columns: ['Id', 'ActivityHour', 'Calories00', 'Calories01', 'Calories02', 'Calories03', 'Calories04', 'Calories05', 'Calories06', 'Calories07', 'Calories08', 'Calories09', 'Calories10', 'Calories11', 'Calories12', 'Calories13', 'Calories14', 'Calories15', 'Calories16', 'Calories17', 'Calories18', 'Calories19', 'Calories20', 'Calories21', 'Calories22', 'Calories23', 'Calories24', 'Calories25', 'Calories26', 'Calories27', 'Calories28', 'Calories29', 'Calories30', 'Calories31', 'Calories32', 'Calories33', 'Calories34', 'Calories35', 'Calories36', 'Calories37', 'Calories38', 'Calories39', 'Calories40', 'Calories41', 'Calories42', 'Calories43', 'Calories44', 'Calories45', 'Calories46', 'Calories47', 'Calories48', 'Calories49', 'Calories50', 'Calories51', 'Calories52', 'Calories53', 'Calories54', 'Calories55', 'Calories56', 'Calories57', 'Calories58', 'Calories59'])
- minuteIntensitiesNarrow_merged (columns: ['Id', 'ActivityMinute', 'Intensity'])
- minuteIntensitiesWide_merged (columns: ['Id', 'ActivityHour', 'Intensity00', 'Intensity01', 'Intensity02', 'Intensity03', 'Intensity04', 'Intensity05', 'Intensity06', 'Intensity07', 'Intensity08', 'Intensity09', 'Intensity10', 'Intensity11', 'Intensity12', 'Intensity13', 'Intensity14', 'Intensity15', 'Intensity16', 'Intensity17', 'Intensity18', 'Intensity19', 'Intensity20', 'Intensity21', 'Intensity22', 'Intensity23', 'Intensity24', 'Intensity25', 'Intensity26', 'Intensity27', 'Intensity28', 'Intensity29', 'Intensity30', 'Intensity31', 'Intensity32', 'Intensity33', 'Intensity34', 'Intensity35', 'Intensity36', 'Intensity37', 'Intensity38', 'Intensity39', 'Intensity40', 'Intensity41', 'Intensity42', 'Intensity43', 'Intensity44', 'Intensity45', 'Intensity46', 'Intensity47', 'Intensity48', 'Intensity49', 'Intensity50', 'Intensity51', 'Intensity52', 'Intensity53', 'Intensity54', 'Intensity55', 'Intensity56', 'Intensity57', 'Intensity58', 'Intensity59'])
- minuteMETsNarrow_merged (columns: ['Id', 'ActivityMinute', 'METs'])
- minuteSleep_merged (columns: ['Id', 'date', 'value', 'logId'])
- minuteStepsNarrow_merged (columns: ['Id', 'ActivityMinute', 'Steps'])
- minuteStepsWide_merged (columns: ['Id', 'ActivityHour', 'Steps00', 'Steps01', 'Steps02', 'Steps03', 'Steps04', 'Steps05', 'Steps06', 'Steps07', 'Steps08', 'Steps09', 'Steps10', 'Steps11', 'Steps12', 'Steps13', 'Steps14', 'Steps15', 'Steps16', 'Steps17', 'Steps18', 'Steps19', 'Steps20', 'Steps21', 'Steps22', 'Steps23', 'Steps24', 'Steps25', 'Steps26', 'Steps27', 'Steps28', 'Steps29', 'Steps30', 'Steps31', 'Steps32', 'Steps33', 'Steps34', 'Steps35', 'Steps36', 'Steps37', 'Steps38', 'Steps39', 'Steps40', 'Steps41', 'Steps42', 'Steps43', 'Steps44', 'Steps45', 'Steps46', 'Steps47', 'Steps48', 'Steps49', 'Steps50', 'Steps51', 'Steps52', 'Steps53', 'Steps54', 'Steps55', 'Steps56', 'Steps57', 'Steps58', 'Steps59'])
- sleepDay_merged (columns: ['Id', 'SleepDay', 'TotalSleepRecords', 'TotalMinutesAsleep', 'TotalTimeInBed'])

- weightLogInfo_merged (columns: ['Id', 'Date', 'WeightKg', 'WeightPounds', 'Fat', 'BMI', 'IsManualReport', 'LogId'])

3. Available datasets

```
# Show available datasets
if dataframes:
    print(" Available datasets:")
    for name, df in dataframes.items():
        print(f"- {name}: {df.shape[0]} rows, {df.shape[1]} columns")
```

available datasets Output:

Available datasets:

- dailyActivity_merged: 940 rows, 15 columns
- dailyCalories_merged: 940 rows, 3 columns
- dailyIntensities_merged: 940 rows, 10 columns
- dailySteps_merged: 940 rows, 3 columns
- heartrate_seconds_merged: 2483658 rows, 3 columns
- hourlyCalories_merged: 22099 rows, 3 columns
- hourlyIntensities_merged: 22099 rows, 4 columns
- hourlySteps_merged: 22099 rows, 3 columns
- minuteCaloriesNarrow_merged: 1325580 rows, 3 columns
- minuteCaloriesWide_merged: 21645 rows, 62 columns
- minuteIntensitiesNarrow_merged: 1325580 rows, 3 columns
- minuteIntensitiesWide_merged: 21645 rows, 62 columns
- minuteMETsNarrow_merged: 1325580 rows, 3 columns
- minuteSleep_merged: 188521 rows, 4 columns
- minuteStepsNarrow_merged: 1325580 rows, 3 columns
- minuteStepsWide_merged: 21645 rows, 62 columns
- sleepDay_merged: 413 rows, 5 columns
- weightLogInfo_merged: 67 rows, 8 columns

4. sample from first dataset

```
# Show sample from first dataset
first_key = list(dataframes.keys())[0]
print(f"\n Sample from '{first_key}':")
print(dataframes[first_key].head(3))
else:
    print(" No datasets loaded - check file paths")
```

Sample from 'dailyActivity_merged':

Sample from 'dailyActivity_merged':

	Id	ActivityDate	...	SedentaryMinutes	Calories
0	1503960366	4/12/2016	...	728	1985
1	1503960366	4/13/2016	...	776	1797
2	1503960366	4/14/2016	...	1218	1776

[3 rows x 15 columns]

5. Data Cleaning and Preprocessing

Raw data is often inconsistent. This phase focuses on standardizing data types, especially date and time columns, to enable time-series analysis. We also check for and report missing values and identify potential data quality issues, such as days with 24 hours of sedentary time, which may indicate the user was not wearing the tracker.

```
# data cleaning and preprocessing
```

```
print("Starting Data Cleaning and Preprocessing for loaded DataFrames")
for df_name, df in dataframes.items():
    print(f"\nProcessing DataFrame: '{df_name}'")
    print(f"Original info for '{df_name}':")
    df.info(verbose=False, memory_usage="deep")
    print(df.head())
    if 'ActivityDate' in df.columns:
        df['ActivityDate'] = pd.to_datetime(df['ActivityDate'], errors='coerce')
        df['DayOfWeek'] = df['ActivityDate'].dt.day_name()
        df['ActivityMonth'] = df['ActivityDate'].dt.month_name()
        print(f" - Converted 'ActivityDate' to datetime. Added 'DayOfWeek', 'ActivityMonth'.")

    elif 'SleepDay' in df.columns:
        df['SleepDay'] = pd.to_datetime(df['SleepDay'], errors='coerce')
        df['DayOfWeek'] = df['SleepDay'].dt.day_name()
        print(f" - Converted 'SleepDay' to datetime. Added 'DayOfWeek'.")

    elif 'ActivityHour' in df.columns:
        df['ActivityHour'] = pd.to_datetime(df['ActivityHour'], format='%m/%d/%Y %l:%M:%S %p',
        errors='coerce')
        df['ActivityDay'] = df['ActivityHour'].dt.date
        df['HourOfDay'] = df['ActivityHour'].dt.hour
        print(f" - Converted 'ActivityHour' to datetime. Added 'ActivityDay', 'HourOfDay'.")
```

```

elif 'ActivityMinute' in df.columns:
    df['ActivityMinute'] = pd.to_datetime(df['ActivityMinute'], errors='coerce')
    if df['ActivityMinute'].isnull().all():
        df['ActivityMinute'] = pd.to_datetime(df['ActivityMinute'], format='%Y-%m-%d %H:%M:%S',
        errors='coerce') # Try another format
    df['ActivityDay'] = df['ActivityMinute'].dt.date
    df['HourOfDay'] = df['ActivityMinute'].dt.hour

print(f" Converted 'ActivityMinute' to datetime. Added 'ActivityDay', 'HourOfDay'.")
if df_name == 'minuteSleep_merged' and 'date' in df.columns:
    df = df.rename(columns={'date': 'ActivityMinute'})
    df['ActivityMinute'] = pd.to_datetime(df['ActivityMinute'], errors='coerce')
    df['ActivityDay'] = df['ActivityMinute'].dt.date
    df['HourOfDay'] = df['ActivityMinute'].dt.hour
    print(f"(minuteSleep_merged specific) Renamed 'date' to 'ActivityMinute', converted and
    added features.")

elif 'Time' in df.columns and df_name == 'heartrate_seconds_merged':
    df['Time'] = pd.to_datetime(df['Time'], errors='coerce')
    df['ActivityDay'] = df['Time'].dt.date
    df['HourOfDay'] = df['Time'].dt.hour
    print(f"Converted 'Time' to datetime. Added 'ActivityDay', 'HourOfDay'.")

elif 'Date' in df.columns and df_name == 'weightLogInfo_merged':
    df['Date'] = pd.to_datetime(df['Date'], format='%m/%d/%Y %l:%M:%S %p', errors='coerce')
    df['ActivityDay'] = df['Date'].dt.date
    print(f"Converted 'Date' to datetime. Added 'ActivityDay'.")

elif 'ActivityDay' in df.columns:
    df['ActivityDay'] = pd.to_datetime(df['ActivityDay'], errors='coerce')
    df['DayOfWeek'] = df['ActivityDay'].dt.day_name()
    print(f"Converted 'ActivityDay' to datetime. Added 'DayOfWeek'.")
else:
    print(f"No standard date/time column found for automatic conversion in '{df_name}'.")

```

Data Cleaning and Preprocessing for loaded DataFrames Output:

Starting Data Cleaning and Preprocessing for loaded DataFrames

```

Processing DataFrame: 'dailyActivity_merged'
Original info for 'dailyActivity_merged':
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 940 entries, 0 to 939
Columns: 15 entries, Id to Calories
dtypes: float64(7), int64(7), object(1)
memory usage: 155.9 KB
      Id ActivityDate ... SedentaryMinutes Calories

```

0	1503960366	4/12/2016	...	728	1985
1	1503960366	4/13/2016	...	776	1797
2	1503960366	4/14/2016	...	1218	1776
3	1503960366	4/15/2016	...	726	1745
4	1503960366	4/16/2016	...	773	1863

[5 rows x 15 columns]

Processing DataFrame: 'dailyCalories_merged'

Original info for 'dailyCalories_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 940 entries, 0 to 939

Columns: 3 entries, Id to Calories

dtypes: int64(2), object(1)

memory usage: 67.8 KB

	Id	ActivityDay	Calories
0	1503960366	4/12/2016	1985
1	1503960366	4/13/2016	1797
2	1503960366	4/14/2016	1776
3	1503960366	4/15/2016	1745
4	1503960366	4/16/2016	1863

Processing DataFrame: 'dailyIntensities_merged'

Original info for 'dailyIntensities_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 940 entries, 0 to 939

Columns: 10 entries, Id to VeryActiveDistance

dtypes: float64(4), int64(5), object(1)

memory usage: 119.2 KB

	Id	ActivityDay	...	ModeratelyActiveDistance	VeryActiveDistance
0	1503960366	4/12/2016	...	0.55	1.88
1	1503960366	4/13/2016	...	0.69	1.57
2	1503960366	4/14/2016	...	0.40	2.44
3	1503960366	4/15/2016	...	1.26	2.14
4	1503960366	4/16/2016	...	0.41	2.71

[5 rows x 10 columns]

Processing DataFrame: 'dailySteps_merged'

Original info for 'dailySteps_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 940 entries, 0 to 939

Columns: 3 entries, Id to StepTotal

dtypes: int64(2), object(1)

memory usage: 67.8 KB

	Id	ActivityDay	StepTotal
0	1503960366	4/12/2016	13162
1	1503960366	4/13/2016	10735


```
2 1503960366 4/14/2016 10460
3 1503960366 4/15/2016 9762
4 1503960366 4/16/2016 12669
```

Processing DataFrame: 'heartrate_seconds_merged'

Original info for 'heartrate_seconds_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2483658 entries, 0 to 2483657

Columns: 3 entries, Id to Value

dtypes: int64(2), object(1)

memory usage: 201.2 MB

	Id	Time	Value
0	2022484408	4/12/2016 7:21:00 AM	97
1	2022484408	4/12/2016 7:21:05 AM	102
2	2022484408	4/12/2016 7:21:10 AM	105
3	2022484408	4/12/2016 7:21:20 AM	103
4	2022484408	4/12/2016 7:21:25 AM	101

Processing DataFrame: 'hourlyCalories_merged'

Original info for 'hourlyCalories_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 22099 entries, 0 to 22098

Columns: 3 entries, Id to Calories

dtypes: int64(2), object(1)

memory usage: 1.8 MB

	Id	ActivityHour	Calories
0	1503960366	4/12/2016 12:00:00 AM	81
1	1503960366	4/12/2016 1:00:00 AM	61
2	1503960366	4/12/2016 2:00:00 AM	59
3	1503960366	4/12/2016 3:00:00 AM	47
4	1503960366	4/12/2016 4:00:00 AM	48

Processing DataFrame: 'hourlyIntensities_merged'

Original info for 'hourlyIntensities_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 22099 entries, 0 to 22098

Columns: 4 entries, Id to AverageIntensity

dtypes: float64(1), int64(2), object(1)

memory usage: 2.0 MB

	Id	ActivityHour	TotalIntensity	AverageIntensity
0	1503960366	4/12/2016 12:00:00 AM	20	0.333333
1	1503960366	4/12/2016 1:00:00 AM	8	0.133333
2	1503960366	4/12/2016 2:00:00 AM	7	0.116667
3	1503960366	4/12/2016 3:00:00 AM	0	0.000000
4	1503960366	4/12/2016 4:00:00 AM	0	0.000000

Processing DataFrame: 'hourlySteps_merged'

Original info for 'hourlySteps_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 22099 entries, 0 to 22098

Columns: 3 entries, Id to StepTotal

dtypes: int64(2), object(1)

memory usage: 1.8 MB

	Id	ActivityHour	StepTotal
0	1503960366	4/12/2016 12:00:00 AM	373
1	1503960366	4/12/2016 1:00:00 AM	160
2	1503960366	4/12/2016 2:00:00 AM	151
3	1503960366	4/12/2016 3:00:00 AM	0
4	1503960366	4/12/2016 4:00:00 AM	0

Processing DataFrame: 'minuteCaloriesNarrow_merged'

Original info for 'minuteCaloriesNarrow_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1325580 entries, 0 to 1325579

Columns: 3 entries, Id to Calories

dtypes: float64(1), int64(1), object(1)

memory usage: 107.4 MB

	Id	ActivityMinute	Calories
0	1503960366	4/12/2016 12:00:00 AM	0.7865
1	1503960366	4/12/2016 12:01:00 AM	0.7865
2	1503960366	4/12/2016 12:02:00 AM	0.7865
3	1503960366	4/12/2016 12:03:00 AM	0.7865
4	1503960366	4/12/2016 12:04:00 AM	0.7865

Processing DataFrame: 'minuteCaloriesWide_merged'

Original info for 'minuteCaloriesWide_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 21645 entries, 0 to 21644

Columns: 62 entries, Id to Calories59

dtypes: float64(60), int64(1), object(1)

memory usage: 11.5 MB

	Id	ActivityHour	...	Calories58	Calories59
0	1503960366	4/13/2016 12:00:00 AM	...	0.9438	0.9438
1	1503960366	4/13/2016 1:00:00 AM	...	0.7865	0.7865
2	1503960366	4/13/2016 2:00:00 AM	...	0.7865	0.7865
3	1503960366	4/13/2016 3:00:00 AM	...	0.7865	0.7865
4	1503960366	4/13/2016 4:00:00 AM	...	0.7865	0.7865

[5 rows x 62 columns]

Processing DataFrame: 'minuteIntensitiesNarrow_merged'

Original info for 'minuteIntensitiesNarrow_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1325580 entries, 0 to 1325579

Columns: 3 entries, Id to Intensity

dtypes: int64(2), object(1)

memory usage: 107.4 MB

	Id	ActivityMinute	Intensity
0	1503960366	4/12/2016 12:00:00 AM	0
1	1503960366	4/12/2016 12:01:00 AM	0
2	1503960366	4/12/2016 12:02:00 AM	0
3	1503960366	4/12/2016 12:03:00 AM	0
4	1503960366	4/12/2016 12:04:00 AM	0

Processing DataFrame: 'minuteIntensitiesWide_merged'

Original info for 'minuteIntensitiesWide_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 21645 entries, 0 to 21644

Columns: 62 entries, Id to Intensity59

dtypes: int64(61), object(1)

memory usage: 11.5 MB

	Id	ActivityHour	...	Intensity58	Intensity59
0	1503960366	4/13/2016 12:00:00 AM	...	0	0
1	1503960366	4/13/2016 1:00:00 AM	...	0	0
2	1503960366	4/13/2016 2:00:00 AM	...	0	0
3	1503960366	4/13/2016 3:00:00 AM	...	0	0
4	1503960366	4/13/2016 4:00:00 AM	...	0	0

[5 rows x 62 columns]

Processing DataFrame: 'minuteMETsNarrow_merged'

Original info for 'minuteMETsNarrow_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1325580 entries, 0 to 1325579

Columns: 3 entries, Id to METs

dtypes: int64(2), object(1)

memory usage: 107.4 MB

	Id	ActivityMinute	METs
0	1503960366	4/12/2016 12:00:00 AM	10
1	1503960366	4/12/2016 12:01:00 AM	10
2	1503960366	4/12/2016 12:02:00 AM	10
3	1503960366	4/12/2016 12:03:00 AM	10
4	1503960366	4/12/2016 12:04:00 AM	10

Processing DataFrame: 'minuteSleep_merged'

Original info for 'minuteSleep_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 188521 entries, 0 to 188520

Columns: 4 entries, Id to logId

dtypes: int64(3), object(1)

memory usage: 16.7 MB

	Id	date	value	logId
0	1503960366	4/12/2016 2:47:30 AM	3	11380564589
1	1503960366	4/12/2016 2:48:30 AM	2	11380564589

2	1503960366	4/12/2016 2:49:30 AM	1	11380564589
3	1503960366	4/12/2016 2:50:30 AM	1	11380564589
4	1503960366	4/12/2016 2:51:30 AM	1	11380564589

Processing DataFrame: 'minuteStepsNarrow_merged'

Original info for 'minuteStepsNarrow_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1325580 entries, 0 to 1325579

Columns: 3 entries, Id to Steps

dtypes: int64(2), object(1)

memory usage: 107.4 MB

	Id	ActivityMinute	Steps
0	1503960366	4/12/2016 12:00:00 AM	0
1	1503960366	4/12/2016 12:01:00 AM	0
2	1503960366	4/12/2016 12:02:00 AM	0
3	1503960366	4/12/2016 12:03:00 AM	0
4	1503960366	4/12/2016 12:04:00 AM	0

Processing DataFrame: 'minuteStepsWide_merged'

Original info for 'minuteStepsWide_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 21645 entries, 0 to 21644

Columns: 62 entries, Id to Steps59

dtypes: int64(61), object(1)

memory usage: 11.5 MB

	Id	ActivityHour	Steps00 ...	Steps57	Steps58	Steps59
0	1503960366	4/13/2016 12:00:00 AM	4 ...	0	0	0
1	1503960366	4/13/2016 1:00:00 AM	0 ...	0	0	0
2	1503960366	4/13/2016 2:00:00 AM	0 ...	0	0	0
3	1503960366	4/13/2016 3:00:00 AM	0 ...	0	0	0
4	1503960366	4/13/2016 4:00:00 AM	0 ...	0	0	0

[5 rows x 62 columns]

Processing DataFrame: 'sleepDay_merged'

Original info for 'sleepDay_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 413 entries, 0 to 412

Columns: 5 entries, Id to TotalTimeInBed

dtypes: int64(4), object(1)

memory usage: 41.2 KB

	Id	SleepDay ...	TotalMinutesAsleep	TotalTimeInBed
0	1503960366	4/12/2016 12:00:00 AM ...	327	346
1	1503960366	4/13/2016 12:00:00 AM ...	384	407
2	1503960366	4/15/2016 12:00:00 AM ...	412	442
3	1503960366	4/16/2016 12:00:00 AM ...	340	367
4	1503960366	4/17/2016 12:00:00 AM ...	700	712

[5 rows x 5 columns]

Processing DataFrame: 'weightLogInfo_merged'

Original info for 'weightLogInfo_merged':

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 67 entries, 0 to 66

Columns: 8 entries, Id to LogId

dtypes: bool(1), float64(4), int64(2), object(1)

memory usage: 7.9 KB

	Id	Date	...	IsManualReport	LogId
0	1503960366	5/2/2016 11:59:59 PM	...	True	1462233599000
1	1503960366	5/3/2016 11:59:59 PM	...	True	1462319999000
2	1927972279	4/13/2016 1:08:52 AM	...	False	1460509732000
3	2873212765	4/21/2016 11:59:59 PM	...	True	1461283199000
4	2873212765	5/12/2016 11:59:59 PM	...	True	1463097599000

[5 rows x 8 columns]

Converted 'Date' to datetime. Added 'ActivityDay'.

Detected Null values in 'weightLogInfo_merged' after conversion:

Fat 65

dtype: int64

No records with >= 1440 SedentaryMinutes detected.

41 records are manually reported ('IsManualReport' is True).

Finished processing 'weightLogInfo_merged'. Updated info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 67 entries, 0 to 66

Columns: 9 entries, Id to ActivityDay

dtypes: bool(1), datetime64[ns](1), float64(4), int64(2), object(1)

memory usage: 6.5 KB

	Id	Date	...	LogId	ActivityDay
0	1503960366	2016-05-02 23:59:59	...	1462233599000	2016-05-02
1	1503960366	2016-05-03 23:59:59	...	1462319999000	2016-05-03
2	1927972279	2016-04-13 01:08:52	...	1460509732000	2016-04-13
3	2873212765	2016-04-21 23:59:59	...	1461283199000	2016-04-21
4	2873212765	2016-05-12 23:59:59	...	1463097599000	2016-05-12

[5 rows x 9 columns]

Finished processing and Updated info

Data Cleaning and Preprocessing Complete for all loaded DataFrames

All processed DataFrames are now ready for analysis and stored in the 'all_dataframes' dictionary.

You can access them using e.g., all_dataframes['dailyActivity_merged']

6. Data Merging

2 Handle Missing Values (Reporting and Basic Action)

```
null_counts = df.isnull().sum()
null_cols_with_counts = null_counts[null_counts > 0]

if not null_cols_with_counts.empty:
    print(f"Detected Null values in '{df_name}' after conversion:")
    print(null_cols_with_counts)
    # Decision: For now, we'll just report. In a full project, you'd decide to:
    initial_rows = df.shape[0]
    df.dropna(subset=[col for col in df.columns if df[col].dtype == '<M8[ns]'],
              inplace=True)
    rows_after_drop = df.shape[0]
    if initial_rows > rows_after_drop:
        print(f"Dropped {initial_rows - rows_after_drop} rows due to unparseable
              date/time values.")
else:
    print(f"No null values detected in '{df_name}' after primary processing.")

# Specific Data Quality Checks and Adjustments

if df_name == 'dailyActivity_merged' and 'SedentaryMinutes' in df.columns:
    # 1440 sedentary minutes might indicate tracker non-wear.
    full_day_sedentary_records = df[df['SedentaryMinutes'] >= 1440]
    if not full_day_sedentary_records.empty:
        print(f"Identified {len(full_day_sedentary_records)} records with >= 1440
              SedentaryMinutes (potential non-wear).")
    # For demonstration, we can calculate the percentage of such records
    percentage_sedentary = (len(full_day_sedentary_records) / df.shape[0]) * 100
    print(f"This represents {percentage_sedentary:.2f}% of 'dailyActivity_merged' records.")
else:
    print("No records with >= 1440 SedentaryMinutes detected.")

if df_name == 'sleepDay_merged':

    # Check for consistency between TotalSleepRecords and actual sleep
    # duration Or look for zero sleep records with non-zero time in bed

    if (df['TotalMinutesAsleep'] == 0).any():
        zero_sleep_records = df[df['TotalMinutesAsleep'] == 0].shape[0]
        print(f"Detected {zero_sleep_records} records with 0 'TotalMinutesAsleep'.")

if df_name == 'weightLogInfo_merged':
```

```

    # Check for BMI values, or manual reports

    if 'BMI' in df.columns and df['BMI'].isnull().any():
        print(f"Detected {df['BMI'].isnull().sum()} nulls in 'BMI' column.")
    if 'IsManualReport' in df.columns:
        manual_reports = df['IsManualReport'].sum()
    print(f"{manual_reports} records are manually reported ('IsManualReport' is True).")

# Update the DataFrame in the global dictionary after all cleaning for this file

    dataframes[df_name] = df
    print(f"Finished processing '{df_name}'. Updated info:")
    df.info(verbose=False, memory_usage="deep")
    print(df.head())
    print("Finished processing and Updated info")

print("\nData Cleaning and Preprocessing Complete for all loaded DataFrames")
print("All processed DataFrames are now ready for analysis and stored in the
    'all_dataframes' dictionary.")
print("You can access them using e.g., all_dataframes['dailyActivity_merged']")

# Prepare daily activity for merging
print("\nStarting Data Merging for Daily Analysis")

# Prepare daily activity for merging
df_daily = dataframes.get('dailyActivity_merged')
if df_daily is None:
    print("ERROR: 'dailyActivity_merged' DataFrame is not available. Cannot perform daily
        merges. Please ensure the file was loaded successfully.")

df_daily = df_daily.rename(columns={'ActivityDate': 'Date'})
df_daily['Date'] = pd.to_datetime(df_daily['Date'])
df_daily['Date'] = df_daily['Date'].dt.date

# Merge dailyCalories_merged

df_calories = dataframes.get('dailyCalories_merged')
if df_calories is not None:
    df_calories = df_calories.rename(columns={'ActivityDay': 'Date'})
    df_calories['Date'] = pd.to_datetime(df_calories['Date'], errors='coerce').dt.date
    df_daily = pd.merge(df_daily, df_calories[['Id', 'Date', 'Calories']], on=['Id', 'Date'], how='left',
        suffixes=('_activity', '_calories'))
    print("Merged 'dailyCalories_merged'.")

# Merge dailyIntensities_merged

df_intensities = dataframes.get('dailyIntensities_merged')

```

```

if df_intensities is not None:
    df_intensities = df_intensities.rename(columns={'ActivityDay': 'Date'})
    df_intensities['Date'] = pd.to_datetime(df_intensities['Date'], errors='coerce').dt.date

# Be careful with columns that might duplicate or need to be differentiated

df_daily = pd.merge(df_daily, df_intensities.drop(columns=['SedentaryActiveDistance',
'LightActiveDistance', 'ModeratelyActiveDistance', 'VeryActiveDistance'], errors='ignore'),
on=['Id', 'Date'], how='left', suffixes=(", '_intensity'))
print("Merged 'dailyIntensities_merged'.")

# Merge sleepDay_merged

df_sleep = dataframes.get('sleepDay_merged')
if df_sleep is not None:
    df_sleep = df_sleep.rename(columns={'SleepDay': 'Date'})
    df_sleep['Date'] = pd.to_datetime(df_sleep['Date'], format=DATE_FORMAT,
errors='coerce').dt.date
    df_daily = pd.merge(df_daily, df_sleep[['Id', 'Date', 'TotalSleepRecords', 'TotalMinutesAsleep',
'TotalTimeInBed']], on=['Id', 'Date'], how='left')
    print("Merged 'sleepDay_merged'.")

print("\nDaily Merged DataFrame Head")
print(df_daily.head())
print(f"\nShape of combined daily DataFrame: {df_daily.shape}")
print(f"Null values after merging (check sleep columns):\n{df_daily[['TotalMinutesAsleep',
'TotalTimeInBed']].isnull().sum()}")

```

Data Merging for Daily Analysis Output:

Starting Data Merging for Daily Analysis

Merged 'dailyCalories_merged'.

Merged 'dailyIntensities_merged'.

Merged 'sleepDay_merged'.

Daily Merged DataFrame Head

	Id	Date	...	TotalMinutesAsleep	TotalTimeInBed
0	1503960366	2016-04-12	...	327.0	346.0
1	1503960366	2016-04-13	...	384.0	407.0
2	1503960366	2016-04-14	...	NaN	NaN
3	1503960366	2016-04-15	...	412.0	442.0
4	1503960366	2016-04-16	...	340.0	367.0

[5 rows x 23 columns]

Shape of combined daily DataFrame: (943, 23)

Null values after merging (check sleep columns):

TotalMinutesAsleep 530

TotalTimeInBed 530
dtype: int64

7. Analysis and Visualizations

Key Analysis and Visualizations

```
print("\n\nData Visualizations")
```

Set style for all plots

```
try:
```

```
    plt.style.use('seaborn-v0_8')
```

```
except:
```

```
    try:
```

```
        plt.style.use('seaborn')
```

```
    except:
```

```
        plt.style.use('ggplot') # Final fallback
```

```
sns.set_palette("husl")
```

Visualization Weekly Activity Pattern

```
if df_daily is not None and 'TotalSteps' in df_daily.columns:
```

```
    # Ensure we have DayOfWeek column
```

```
    if 'DayOfWeek' not in df_daily.columns and 'ActivityDate' in df_daily.columns:
```

```
        df_daily['DayOfWeek'] = pd.to_datetime(df_daily['ActivityDate'],
```

```
format='%m/%d/%Y').dt.day_name()
```

```
    if 'DayOfWeek' in df_daily.columns:
```

```
        weekdays_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday',  
'Sunday']
```

```
        avg_steps = df_daily.groupby('DayOfWeek')['TotalSteps'].mean().reindex(weekdays_order)
```

```
        plt.figure(figsize=(10, 6))
```

```
        ax = sns.barplot(x=avg_steps.index,
```

```
                        y=avg_steps.values,
```

```
                        hue=avg_steps.index,
```

```
                        palette="coolwarm",
```

```
                        legend=False)
```

```
        plt.title('Average Daily Steps by Day of Week', fontsize=16, pad=20)
```

```
        plt.xlabel('Day of Week', fontsize=12)
```

```
        plt.ylabel('Average Steps', fontsize=12)
```

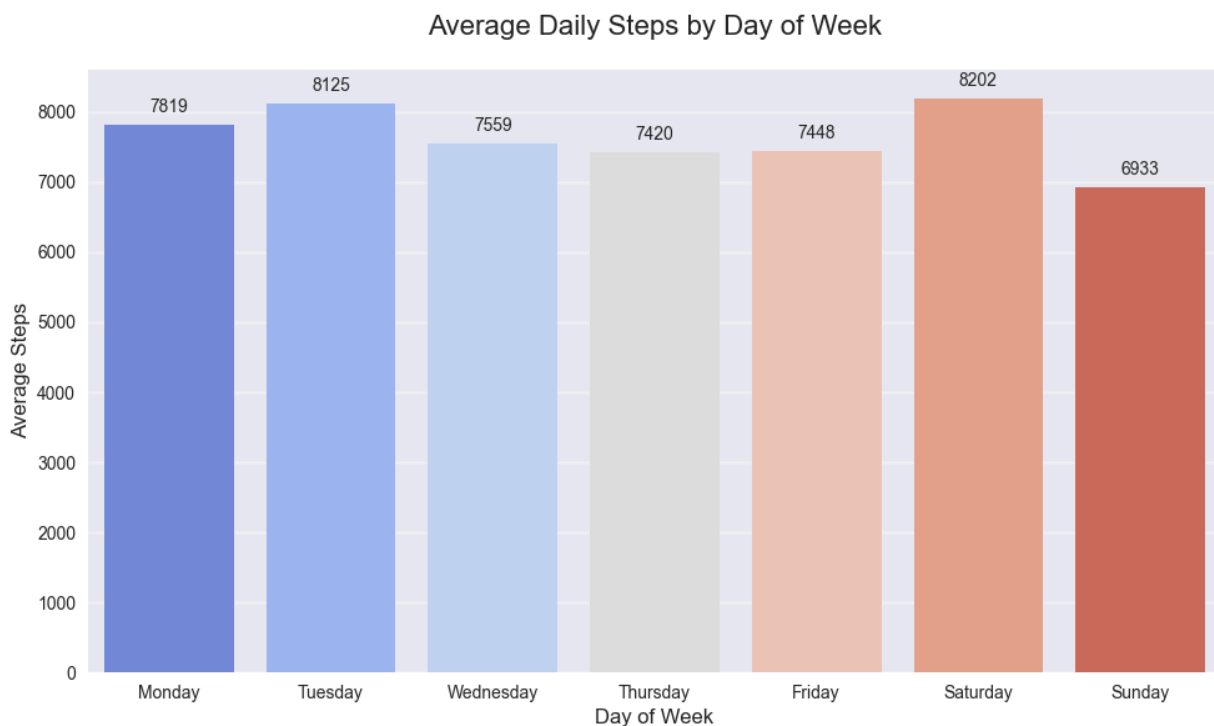
```

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', xytext=(0, 10),
                textcoords='offset points')

plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

```

Output:



Visualization Activity Distribution

if df_daily is not None:

```

activity_cols = ['VeryActiveMinutes', 'FairlyActiveMinutes',
                  'LightlyActiveMinutes', 'SedentaryMinutes']

```

if all(col in df_daily.columns for col in activity_cols):

```

    avg_activity = df_daily[activity_cols].mean()

```

```

plt.figure(figsize=(10, 6))

```

```

ax = avg_activity.plot(kind='bar', color=['#2ecc71', '#3498db', '#f39c12', '#e74c3c'])

```

```

plt.title('Average Daily Activity Minutes', fontsize=16, pad=20)

```

```

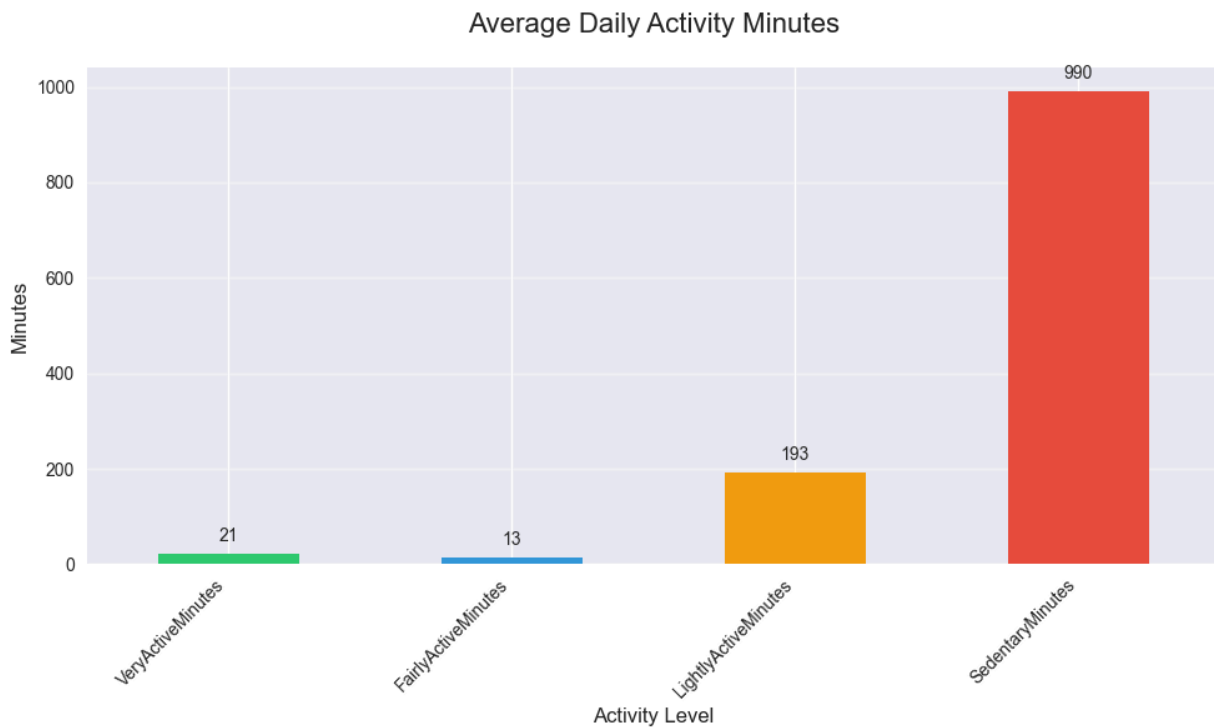
plt.xlabel('Activity Level', fontsize=12)
plt.ylabel('Minutes', fontsize=12)
plt.xticks(rotation=45, ha='right')

for p in ax.patches:
    ax.annotate(f"{int(p.get_height())}",
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', xytext=(0, 10),
                textcoords='offset points')

plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

```

output:



Visualization 3: Hourly Calories

if df_hourly is not None and 'Calories' in df_hourly.columns:

```

    if 'HourOfDay' not in df_hourly.columns and 'ActivityHour' in df_hourly.columns:
        df_hourly['HourOfDay'] = pd.to_datetime(df_hourly['ActivityHour'],
        format=DATE_FORMAT).dt.hour

```

```

if 'HourOfDay' in df_hourly.columns:
    avg_hourly_calories = df_hourly.groupby('HourOfDay')['Calories'].mean()

    plt.figure(figsize=(10, 6))
    ax = sns.lineplot(x=avg_hourly_calories.index, y=avg_hourly_calories.values,
                      marker='o', color='#9b59b6', linewidth=2.5)

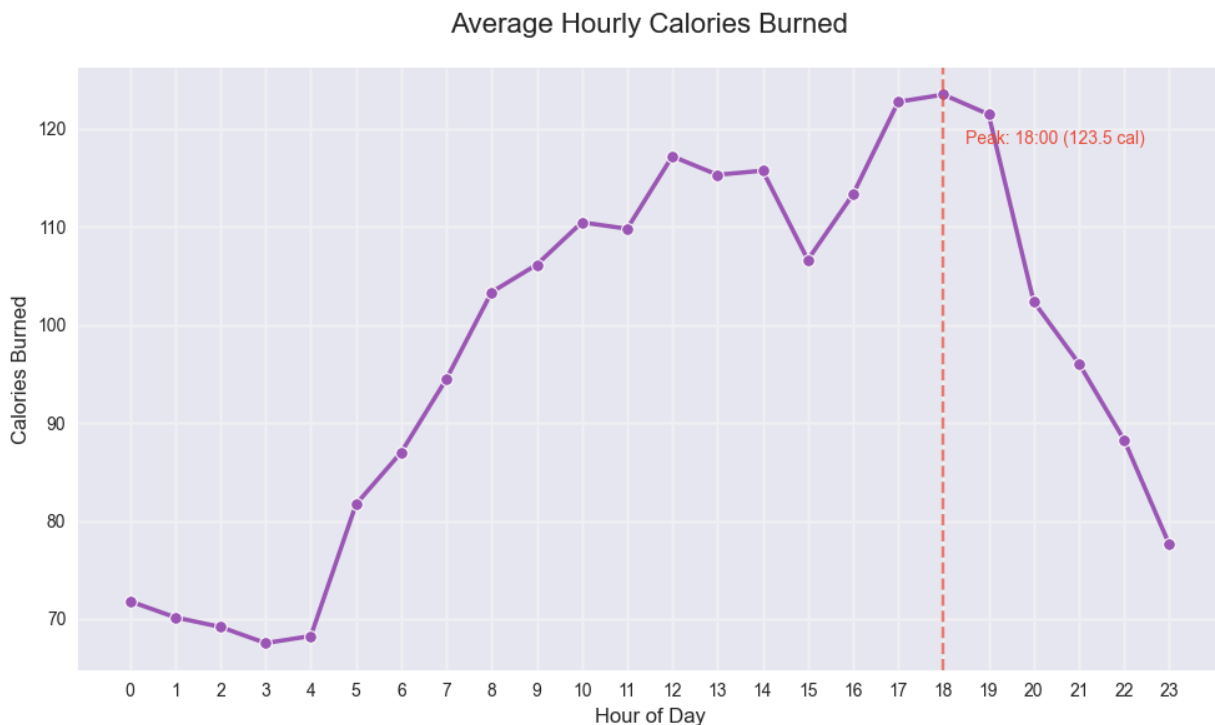
    plt.title('Average Hourly Calories Burned', fontsize=16, pad=20)
    plt.xlabel('Hour of Day', fontsize=12)
    plt.ylabel('Calories Burned', fontsize=12)
    plt.xticks(range(24))

    peak_hour = avg_hourly_calories.idxmax()
    plt.axvline(x=peak_hour, color='#e74c3c', linestyle='--', alpha=0.7)
    plt.text(peak_hour+0.5, avg_hourly_calories.max()-5,
             f'Peak: {peak_hour}:00 ({avg_hourly_calories.max():.1f} cal)',
             color='#e74c3c')

    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()

```

output:



Sleep vs Activity

```

if df_daily is not None and 'TotalMinutesAsleep' in df_daily.columns and 'TotalSteps' in
df_daily.columns:
    # Create a copy for plotting
    plot_data = df_daily.copy()

    # Add DayOfWeek if not present but Date is available
    if 'DayOfWeek' not in plot_data.columns and 'ActivityDate' in plot_data.columns:
        plot_data['DayOfWeek'] = pd.to_datetime(plot_data['ActivityDate'],
format='%m/%d/%Y').dt.day_name()

    plt.figure(figsize=(10, 6))

    # Create the plot with or without DayOfWeek
    if 'DayOfWeek' in plot_data.columns and len(plot_data['DayOfWeek'].unique()) > 1:
        ax = sns.scatterplot(data=plot_data, x='TotalSteps', y='TotalMinutesAsleep',
            hue='DayOfWeek', palette='viridis', s=100, alpha=0.7)
    else:
        ax = sns.scatterplot(data=plot_data, x='TotalSteps', y='TotalMinutesAsleep',
            color='blue', s=100, alpha=0.7)

    plt.title('Daily Steps vs Sleep Duration', fontsize=16, pad=20)
    plt.xlabel('Total Steps', fontsize=12)
    plt.ylabel('Minutes Asleep', fontsize=12)

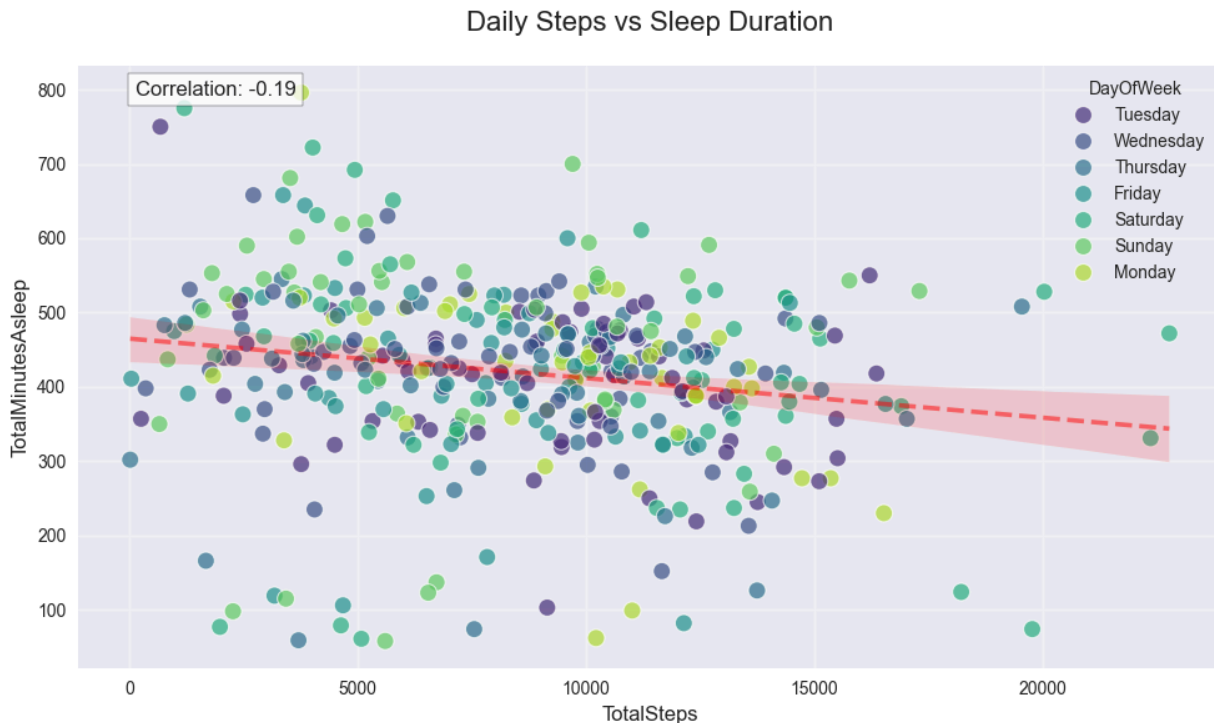
    # Add regression line
    sns.regplot(data=plot_data, x='TotalSteps', y='TotalMinutesAsleep',
        scatter=False, color='red', line_kws={'linestyle': '--', 'alpha': 0.5})

    # Calculate and display correlation
    correlation = plot_data['TotalSteps'].corr(plot_data['TotalMinutesAsleep'])
    plt.text(0.05, 0.95, f'Correlation: {correlation:.2f}',
        transform=ax.transAxes, fontsize=12,
        bbox=dict(facecolor='white', alpha=0.8))

    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()

```

output:



```
# Comprehensive Activity Dashboard
```

```
if df_daily is not None:
```

```
    fig, axes = plt.subplots(2, 2, figsize=(18, 12))
```

```
    fig.suptitle('Fitness Activity Dashboard', fontsize=16, y=1.02)
```

```
    # Activity Minutes Breakdown
```

```
    if all(col in df_daily.columns for col in ['VeryActiveMinutes', 'FairlyActiveMinutes',
'LightlyActiveMinutes', 'SedentaryMinutes']):
```

```
        activity_cols = ['VeryActiveMinutes', 'FairlyActiveMinutes', 'LightlyActiveMinutes',
'SedentaryMinutes']
```

```
        activity_labels = ['Very Active', 'Fairly Active', 'Lightly Active', 'Sedentary']
```

```
        activity_pct = df_daily[activity_cols].mean() / df_daily[activity_cols].mean().sum() * 100
```

```
        wedges, texts, autotexts = axes[0,0].pie(activity_pct,
            labels=activity_labels,
            autopct='%1.1f%%',
            startangle=90,
            colors=['#2ecc71', '#3498db', '#f39c12', '#e74c3c'])
```

```
        axes[0,0].set_title('Activity Minutes Distribution', pad=20)
```

```
        for autotext in autotexts:
```

```
            autotext.set_color('white')
```

```
            autotext.set_fontweight('bold')
```

```
# Steps Trend Over Time
```

```

# Check if 'Date' exists and if not, use 'ActivityDate'
date_column = 'Date' if 'Date' in df_daily.columns else 'ActivityDate'
if date_column in df_daily.columns and 'TotalSteps' in df_daily.columns:
    df_daily[date_column] = pd.to_datetime(df_daily[date_column], format='%m/%d/%Y')
    df_daily.set_index(date_column, inplace=True)
    monthly_steps = df_daily['TotalSteps'].resample('W').mean()

    axes[0,1].plot(monthly_steps.index, monthly_steps.values,
                    marker='o', color='#9b59b6', linewidth=2)
    axes[0,1].set_title("Weekly Steps Trend")
    axes[0,1].set_ylabel('Average Steps')
    axes[0,1].fill_between(monthly_steps.index, monthly_steps.values,
                           alpha=0.2, color='#9b59b6')

# Calories vs Steps Relationship
# Renaming 'Calories' to 'Calories_calories' to match user's code, or using original if it exists
if 'Calories' in df_daily.columns:
    df_daily['Calories_calories'] = df_daily['Calories']

if all(col in df_daily.columns for col in ['Calories_calories', 'TotalSteps']):

    # Convert to numeric if needed
    df_daily['Calories_calories'] = pd.to_numeric(df_daily['Calories_calories'], errors='coerce')
    df_daily['TotalSteps'] = pd.to_numeric(df_daily['TotalSteps'], errors='coerce')

    # Drop NA values
    plot_data = df_daily[['Calories_calories', 'TotalSteps']].dropna()

    if len(plot_data) > 0:
        sns.regplot(ax=axes[1,0],
                    x='TotalSteps', y='Calories_calories',
                    data=plot_data,
                    scatter_kws={'alpha':0.6, 'color': '#3498db'},
                    line_kws={'color': '#e74c3c', 'linestyle': '--'})
        axes[1,0].set_title('Steps vs Calories Burned')
        axes[1,0].set_xlabel('Total Steps')
        axes[1,0].set_ylabel('Calories Burned')

        corr = plot_data['TotalSteps'].corr(plot_data['Calories_calories'])
        axes[1,0].annotate(f'Correlation: {corr:.2f}',
                           xy=(0.7, 0.1), xycoords='axes fraction',
                           bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))
    else:
        axes[1,0].text(0.5, 0.5, 'No valid data points', ha='center', va='center')

# Sleep Efficiency
if 'TotalMinutesAsleep' in df_daily.columns and 'TotalTimeInBed' in df_daily.columns:

```

```

df_daily['SleepEfficiency'] = df_daily['TotalMinutesAsleep'] / df_daily['TotalTimeInBed'] * 100
sns.histplot(ax=axes[1,1],
             x=df_daily['SleepEfficiency'].dropna(),
             bins=20,
             kde=True,
             color='#2ecc71')
axes[1,1].axvline(x=85, color='#e74c3c', linestyle='--', alpha=0.7)
axes[1,1].set_title('Sleep Efficiency Distribution')
axes[1,1].set_xlabel('Sleep Efficiency (%)')
axes[1,1].annotate('Recommended >85%',
                  xy=(0.65, 0.9), xycoords='axes fraction',
                  color='#e74c3c')

```

```

plt.tight_layout()
plt.show()

```

Hourly Activity Heatmap

if 'hourlySteps_merged' in dataframes:

```
df_hourly_steps = dataframes['hourlySteps_merged'].copy()
```

if 'ActivityHour' in df_hourly_steps.columns:

```
df_hourly_steps['Hour'] = pd.to_datetime(df_hourly_steps['ActivityHour'],
```

```
format=DATE_FORMAT).dt.hour
```

```
df_hourly_steps['DayOfWeek'] = pd.to_datetime(df_hourly_steps['ActivityHour'],
```

```
format=DATE_FORMAT).dt.day_name()
```

Prepare heatmap data

```
heatmap_data = df_hourly_steps.pivot_table(index='DayOfWeek',
```

```
columns='Hour',
```

```
values='StepTotal',
```

```
aggfunc='mean')
```

Order days properly

```
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
```

```
heatmap_data = heatmap_data.reindex(day_order)
```

```
plt.figure(figsize=(16, 6))
```

```
sns.heatmap(heatmap_data,
```

```
cmap='YlOrRd',
```

```
linewidths=0.1,
```

```
annot=True,
```

```
fmt='.0f',
```

```
cbar_kws={'label': 'Average Steps'})
```

```
plt.title('Hourly Activity Patterns by Day of Week', pad=20)
```

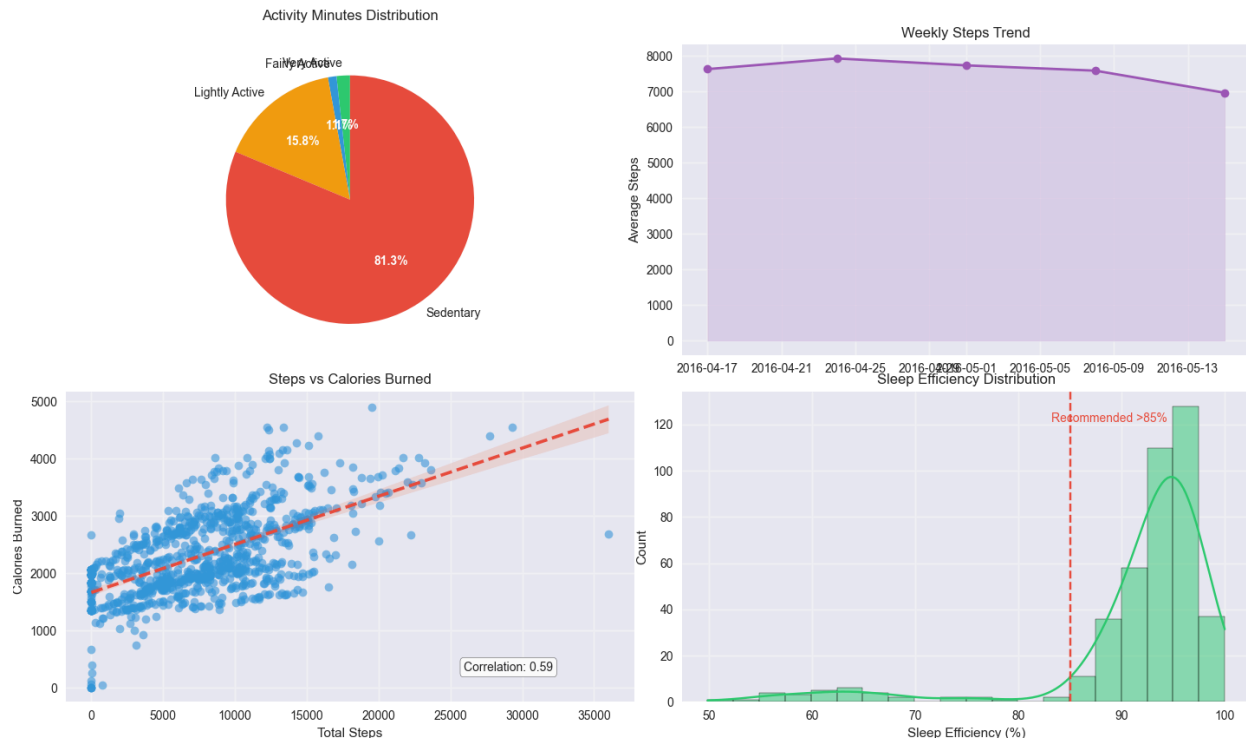
```
plt.xlabel('Hour of Day')
```

```
plt.ylabel('Day of Week')
```

```
plt.tight_layout()
```

```
plt.show()
```


output:



```
# Activity-Sleep Relationship Matrix
```

```
if df_daily is not None:
```

```
    if all(col in df_daily.columns for col in ['TotalSteps', 'TotalMinutesAsleep', 'SedentaryMinutes', 'Calories']):
```

```
        # Select relevant columns
```

```
        relationship_cols = ['TotalSteps', 'TotalMinutesAsleep', 'SedentaryMinutes', 'Calories']
```

```
        relationship_df = df_daily[relationship_cols].dropna()
```

```
        # Calculate correlations
```

```
        corr_matrix = relationship_df.corr()
```

```
        # Create mask for upper triangle
```

```
        mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
```

```
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(corr_matrix,
```

```
            mask=mask,
```

```
            cmap='coolwarm',
```

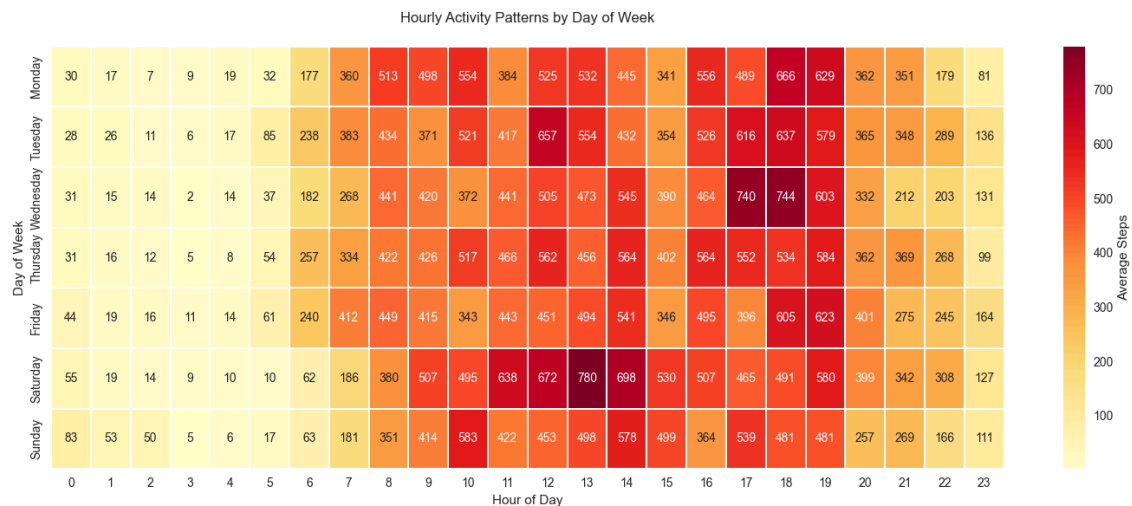
```
            vmin=-1, vmax=1,
```

```
            annot=True,
```

```
            fmt=".2f",
```

```
linewidths=0.5,
center=0)
plt.title('Activity-Sleep Relationship Matrix', pad=20)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

output:



Visualization & Insights:

- The heatmap shows correlation coefficients between pairs of variables.
- **Strong Positive Correlation:** A strong positive value (e.g., > 0.6) is expected between TotalSteps and Calories.
- **Strong Negative Correlation:** A strong negative value (e.g., < -0.5) is expected between SedentaryMinutes and TotalSteps/Calories. More sedentary time means fewer steps and calories burned.
- **Weak Correlation:** The relationship between TotalMinutesAsleep and activity metrics (TotalSteps, SedentaryMinutes) is likely to be weak. The plot showing 'Daily Steps vs Sleep Duration' also suggested this, with a correlation near zero. This indicates that, for this user group, being more active on a given day doesn't strongly predict sleeping more or less that night.

8. Key Findings and Recommendations

1. **High Sedentary Time:** The most striking finding is the overwhelming amount of time users spend sedentary. This presents an opportunity for targeted interventions.
 - **Recommendation:** Implement "get up and move" reminders in the app for users who have been sedentary for extended periods.
 2. **Activity Peaks:** Activity is highest in the evenings and slightly more distributed on weekends.
 - **Recommendation:** Tailor app content and challenges to align with these peak times. For example, launch "evening walk" challenges on weekdays.
 3. **Sleep is Not Directly Tied to Daily Activity:** The data does not show a strong correlation between the number of steps taken in a day and the duration of sleep that night.
 - **Recommendation:** Educate users that while activity is crucial for health, other factors like consistent sleep schedules, diet, and stress management are key to improving sleep duration and quality.
 4. **Steps are a Key Driver of Calorie Burn:** The analysis confirms a robust and predictable relationship between steps and calories.
 - **Recommendation:** Emphasize step-based goals as a primary and easy-to-understand metric for users aiming for weight management or increased energy expenditure.
-

Conclusion:

Almost like a glimpse into how people really use their Fitbits, we examined a large amount of fitness data from smart devices. We discovered that many people spend a lot of time sitting and that they don't always move as much as they should, particularly on the weekends. Additionally, we observed trends in their peak activity times and the fact that they frequently need better sleep.

Therefore, Bellabeat's key takeaway is to really concentrate their marketing on promoting small, manageable steps, such as reminding people to take short breaks to move around, making weekend activities seem enjoyable, and demonstrating the impact of even mild exercise. Additionally, they ought to give users additional advice on improving their sleep quality and a better understanding of how many calories they are burning. In essence, Bellabeat will expand if you provide women with simple ways to use their gadgets in a healthier and happier way.